Transforms All the Way Down — Automatic Online Distributional Forecasting by Conjugation

Peter Cotton
Microprediction
peter.cotton@microprediction.com
https://skaters.microprediction.org

July 6, 2026

Abstract. The Python package **skaters** is an online, distributional, univariate time-series forecaster built by *conjugation*: an invertible transform wraps an inner forecaster — pushing each observation forward, forecasting in the transformed space, and pulling the whole predictive distribution back through the inverse. Because the inner forecaster is itself a conjugation, models nest *transforms all the way down* onto a single distributional leaf, and an ensemble is just another forecaster that combines forecasters. The leaf fits its shape by optimising a proper scoring rule, so the objective is a choice rather than a fixed property of the method: pointed at likelihood the recursion models the conditional density, pointed at CRPS it is competitive with the CRPS specialists on their own metric — fitting and tail-shaping are two settings of one machine, not two machines. The library is written twice, in pure Python and in zero-dependency JavaScript that agrees to within 10^{-6}, so a model runs unchanged on a server or in a browser. **skaters** was first envisaged as a *collection* of forecasters differing by prior; the study reported here pushed the other way, collapsing that collection into a single forecast function with no exposed tuning parameters, which we call `laplace` after the weak (indifference) prior it places over its candidate pool. Positioned as a general-purpose forecaster for economic and other non-price time series, it leads the per-series held-out log-likelihood race against classical, neural, and pretrained foundation-model baselines on FRED series; on asset-price and return series a **GARCH**-t model remains the better choice, and we report that split rather than average it away.

Keywords: probabilistic forecasting, time series, proper scoring rules, conformal prediction, online learning, Python, JavaScript, **skaters**.

1 Introduction

Most deployed forecasters return a point and, perhaps, an interval bolted on afterwards. Yet the quantity a decision-maker actually needs is the *predictive distribution*: the mean for an estimate, the spread for risk, the tails for position sizing, and a density for likelihood-based evaluation and combination. Two strands of recent work take distributions seriously but each gives something up.

Classical state-space and exponential-smoothing models [Hyndman et al., 2002, Hyndman and Khandakar, 2008] are distributional but rarely *online and composable*: adding a transform usually means re-deriving the filter. Conditional-heteroscedasticity models [Engle, 1982, Bollerslev, 1986] capture the heavy tails and volatility clustering of financial data, but commit to a single parametric form and are refit in batch. At the other extreme, conformal prediction [Vovk et al., 2005, 2019] is distribution-free and online, but a conformal predictive system outputs a cumulative distribution function, not a density. Its finite-sample guarantee is *marginal coverage* — not CRPS optimality — and its native output (a set, interval, quantile family, or CDF) carries no unambiguous Lebesgue density, so log-likelihood scoring requires additional smoothing choices on which the resulting

likelihood then depends; an unsmoothed empirical conformal CDF assigns zero density — and hence $-\infty$ log-likelihood — to any value outside the observed residual range. Modern neural and foundation models [Salinas et al., 2020, Ansari et al., 2024, Das et al., 2024, Woo et al., 2024] are powerful distributional learners, but are built for global cross-series training on hardware accelerators, not for a single univariate stream running in a browser.

skaters takes a third route. Every node — transform, ensemble, or leaf — is a function `(y, state) -> ([Dist], state)` that consumes one observation and emits a list of `Dist` objects, one per forecast horizon, where a `Dist` is a weighted Gaussian mixture. Because the type is closed under composition, an entire model is a stack of invertible transforms — transforms all the way down — with one distributional leaf at the bottom, and ensembles are simply skaters that combine other skaters. This single abstraction yields three contributions developed in turn:

1. *A pluggable objective* (Sections 4–5). The leaf fits its mixture weights by a proper scoring rule. Pointed at log-likelihood it models the conditional distribution directly; with the terminal leaf instead pointed at CRPS while the likelihood-weighted trunk is unchanged, the same forecaster becomes competitive with a CRPS specialist on CRPS and stronger on likelihood — the objective is a knob at the base of the recursion, not a property of the method.

2. *Distributional structure that survives composition* (Sections 6–8): a mean-preserving lattice projection that places atoms on the exact values a series revisits without moving the ensemble mean and vanishes on continuous data; an online coordinate search over whether a series is simple in a log, root, or linear coordinate; composable volatility and mean-reversion transforms (a **GARCH**-t leaf, an Ornstein–Uhlenbeck group) that specialise the one forecaster without adding a second; and a multi-scale granularity mixture (Section 9) that extends the same likelihood weighting to the sampling clock itself for multi-step horizons.

3. *A benchmark against twelve distributional baselines* (Section 10), chosen to be difficult to beat on each metric: the heavy-tail state of the art (**GARCH**-t, both **arch** and **rugarch**), the reference R forecasters (the real Hyndman **auto.arima**, **thetaf**, Svetunkov's **ADAM**, **nnetar**), and **AutoARIMA** paired with split- and adaptive-conformal residuals plus conformal seasonal pools — the CRPS-optimised methods hardest to beat on that metric. Every method is scored on held-out log-likelihood and CRPS through a single, identical `Dist`-based protocol.

The unifying premise is the No-Free-Lunch theorem [Wolpert and Macready, 1997] taken seriously: every "method" is a prior, and averaged over all series none dominates. The right response is not to pick one but to make the priors explicit and weight them online: each candidate acts as an *operational prior* over series behaviour, and likelihood weighting limits the practical cost of a poor one by draining its weight as evidence accumulates.

No-Free-Lunch also fixes the scope of our claim. We position **skaters** as a general-purpose forecaster for economic and other non-price series, and that is where the benchmark of Section 10 is run. Asset-price and return series are the hard case — but not because of heavy tails, which the scale-mixture leaf (Section 5) already models. They are hard because they are close to *martingales*: the conditional mean carries almost no signal, so what forecasting skill remains lives in the conditional *variance*, where the dedicated recursion of a **GARCH**$(1,1)$-t still tracks volatility clustering better than our online scale. There we do not claim to win, recommend **GARCH**-t, and report the result as a split rather than dilute it into a pooled average. The distinction is not the price label

but the martingality axis itself (Section 10.2): prices are merely its random-walk-heavy extreme, and the **GARCH**-t contest is governed by where a series falls along it. The library's conditional-variance building blocks (Section 8) narrow but do not close that gap (Table 3: the **GARCH**-t leaf halves the paired log-score deficit on 2,500 price series); matching **GARCH**-t on price series with an online, dependency-free recursion is future work, not a present claim.

2 Using the package

skaters is pure Python with no required dependencies (`pip install skaters`); the JavaScript twin lives in the same repository and loads as an ES module. The entire public surface is one factory and one value type:

```
from skaters import laplace

f = laplace(k=20)          # the general forecaster; multi-scale at k > 1
state = None
for y in stream:
    dists, state = f(y, state)     # list[Dist], one per horizon 1..k
    d = dists[0]
    d.mean, d.std              # point forecast and spread
    d.quantile(0.975)          # tail quantile
    d.logpdf(y), d.crps(y)     # proper-score a realised value
```

Every model in the paper is this call with at most one knob changed: `laplace(k, objective="likelihood")` repoints the terminal leaf (Section 5), `laplace(k, sticky=False)` disables the lattice projection (Section 6), `laplace(k, leaf=garch_leaf)` swaps in the conditional-variance leaf (Section 8), and `laplace(k, scales=[1])` opts out of the multi-scale mixture (Section 9). The building blocks (`conjugate`, the transforms, the leaves, the ensembles) are exported for composition but no benchmark below uses them directly.

The two recursions, exactly. Writing T for a transform with state s_t (Section 3) and g for the inner skater, *conjugation* is

$$\begin{aligned}(z_t,\ s_{t+1}) &= T.\texttt{forward}(y_t,\ s_t)\\ (D'_{1:k},\ u_{t+1}) &= g(z_t,\ u_t)\\ D_{1:k} &= T.\texttt{inverse\_k}(D'_{1:k},\ s_{t+1}),\end{aligned}$$

and the *terminal-leaf ensemble* over candidates $g_1, \dots, g_n$ with depths d_i updates, per observation,

$$\begin{aligned}&\ell_i \leftarrow \gamma\,\ell_i + \eta \log p_i(y_t) - c\,d_i, \qquad w_i \propto e^{\ell_i},\\ &\hat{\mu}_h = \textstyle\sum_i w_i\,\mu_{i,h}, \qquad r_t = y_t - \hat{\mu}_1^{(t-1)}, \qquad D_h = \texttt{leaf}_h(r_t) \text{ shifted by } \hat{\mu}_h,\end{aligned}$$

with $\gamma = 0.99$ (forgetting), $\eta = 0.8$ (learning rate), $c = 0.005$ (per-depth complexity penalty), and p_i the candidate's resolved one-step predictive. These constants are fixed defaults, not exposed parameters; they are the same for every series in every experiment below.

Reproducibility. Section 13 lists the exact commands; every table is regenerated by one harness invocation from the committed result files, and the Python/JavaScript parity suite (roughly 100,000 probe values at 10^{-6}) runs in CI.

3 The `Dist` abstraction and composable transforms

A `Dist` is a weighted mixture $\sum_i w_i \mathcal{N}(\mu_i, \sigma_i^2)$ with $\sigma_i > 0$ and $\sum_i w_i = 1$. It exposes `mean`, `std`, `pdf`, `cdf`, `logpdf`, `quantile`, and a closed-form `crps`, together with the affine maps (`shift`, `scale`, `affine`) and a moment-matching `prune` needed to propagate it through transform inverses and to bound component growth in ensembles. The Gaussian mixture is a deliberate choice: it is closed under affine maps and mixing, admits a closed-form CRPS through the elementary identity $\mathbb{E}\,|\mathcal{N}(m, s^2)| = 2s\,\phi(m/s) + m\,(2\Phi(m/s) - 1)$, and approximates heavy tails arbitrarily well as a scale mixture.

A *transform* is a stateful causal map

$$(z_t,\ s_{t+1}) \ =\ T(y_t,\ s_t),$$

whose `forward` consumes one observation and the stored state and emits one transformed value, paired with an `inverse_k` that reconstructs predictive distributions for future y's from k `Dists` in the transformed space, *conditional on the current transform state* s_t. Several of these maps (differencing, standardisation, autoregression) are not scalar bijections on $\mathbb{R}$; they are invertible as maps on paths given the stored state, which is exactly what `inverse_k` receives. Differencing, exponential smoothing, drift, Holt's linear trend, fractional differencing, autoregression (via online recursive least squares), **GARCH**-style scaling, seasonal differencing, power and Yeo–Johnson coordinates [Box and Cox, 1964, Yeo and Johnson, 2000], and standardisation are all transforms. *Conjugation* composes a transform with an inner skater,

$$y \ \xrightarrow{\ T\ }\ y' \ \to\ \text{inner} \ \to\ D' \ \xrightarrow{\ T^{-1}\ }\ D,$$

and because every step is online and the inverse propagates the full mixture, multi-step uncertainty accumulates through the inverse. One caveat is stated once and holds throughout: the emitted list contains one *marginal* predictive per horizon, not a joint law over the path. Linear inverses propagate means exactly and propagate variances under the library's recursive independent-innovation convention — the differenced inverse, for example, adds the marginal innovation variances, $\sum_h \sigma_h^2$, which is exact when future transformed innovations are conditionally uncorrelated and an approximation otherwise. The inner skater is unconstrained: it may itself be a conjugation, or an ensemble of conjugations, so a model is *transforms all the way down*, bottoming out at a single distributional leaf — the one non-transform in the stack.

Structurally this is the recipe of a normalizing flow [Rezende and Mohamed, 2015] — a base density pushed through invertible maps — but the regime is inverted. A flow *learns* its coupling layers by stochastic gradient descent over a batch; here the transforms are *fixed and analytically invertible* as scalar maps (for nonlinear coordinates the *pushforward of the predictive* uses a delta-method linearisation, Section 7), their few parameters adapting *online* by score-driven, Tweedie-style updates [Creal et al., 2013] one observation at a time, in $O(1)$, with no training phase and no gradients. The base is a weighted mixture with a pluggable proper-scoring objective, not a fixed Gaussian. It is a flow's compositional structure carried into a streaming, closed-form, univariate-forecasting regime.

Numerical note. Mixture variance is computed in centred form, $\mathrm{Var} = \sum_i w_i\,(\sigma_i^2 + (\mu_i - \bar{\mu})^2)$, rather than $\mathbb{E}[X^2] - \mathbb{E}[X]^2$. The latter catastrophically cancels when a tight component sits at a large mean, silently producing $\sigma = 0$ and an invalid predictive. Running moments use Welford's recurrence [Welford, 1962].

4 The terminal-leaf ensemble: modelling the mean

Bayesian model averaging over a heterogeneous pool [Hoeting et al., 1999, Raftery et al., 2005] combines full predictive distributions. In this implementation that dilutes the very shape the leaf is for: with dozens of candidates the pooled mixture is dominated by the dispersion of the candidate *means* (and by component pruning), so a carefully shaped heavy-tailed leaf inside the pool has little influence on the output tail. We therefore use the candidate models only as mean forecasters, weight them online by predictive likelihood (with a learning-rate shrinkage and a per-depth complexity penalty, in the spirit of gradient boosting), combine their means, and model the distribution of the combined residual with a single terminal leaf:

$$y \;\to\; \hat{\mu} = \sum_i w_i\,\hat{\mu}_i, \qquad r = y - \hat{\mu} \;\to\; \text{leaf} \;\to\; D, \qquad \hat{D} = D \text{ shifted by } \hat{\mu}.$$

Because exactly one leaf reaches the output, its shape — heavy tails, learned online — survives undiluted. The trunk is a likelihood-weighted ensemble whose only job is to get the conditional mean right; the shaping of the distribution is left to the base of the recursion. The candidate pool spans exponential smoothing, differencing, drift, Holt's linear trend, autoregression, grouped long-lag autoregression, fractional differencing, a seasonal block, a Yeo–Johnson coordinate grid (Section 7), and a fast/slow two-systems block.

5 Metric-agnostic leaves: a pluggable base

The leaf is itself a fixed Gaussian *scale mixture* — components $\mathcal{N}\big(0, (c_k\,\hat{\sigma})^2\big)$ over a fixed log-spaced dictionary $\{c_k\}$ of standard-deviation multipliers, with $\hat{\sigma}$ an exponentially weighted scale — whose only learned quantities are the mixture weights $\{w_k\}$ on the simplex. Student-t laws are particular Gaussian scale mixtures (an inverse-gamma mixing law over the variance), and the finite log-spaced dictionary approximates such heavy-tailed scale mixtures while remaining a plain `Dist`. The weights are fit online by a proper scoring rule [Gneiting and Raftery, 2007]:

- `likelihood` — recency-weighted expectation–maximisation on the component responsibilities;
- `crps` — exponentiated-gradient descent on the simplex minimising the closed-form mixture CRPS [Matheson and Winkler, 1976]; the gradient is exact, since the mixture CRPS is a finite sum of $\mathbb{E}\,|\mathcal{N}(m, s^2)|$ terms.

Because the leaf is where the whole conjugation bottoms out, it is exactly where a downstream metric should get its vote. The trunk's job is to model, and the right objective for modelling is likelihood, a proper, tail-sensitive score that is, up to a constant, the Kelly log-growth criterion [Kelly, 1956]; corrupting it to chase CRPS would surrender the density, as conformal systems do by construction. Pointing only the terminal leaf at CRPS shapes the residual tail without touching the model above it. Section 11 shows this configuration improving on a pure-likelihood policy on CRPS and on likelihood alike: a CRPS-fit leaf is more outlier-robust, so it even generalises slightly better in log-likelihood.

6 The lattice projection

Administrative and grid-quoted series — policy rates, posted prices, pegged exchange rates — revisit a small set of exact values, and a continuous predictive density cannot place mass there. We add a *projection*: a recency-weighted frequency table of exact values, with near-Dirac atoms on the values revisited above a noise floor, each carrying its observed frequency as probability. The atom is mean-preserving: the continuous part is recentred so the atoms add mass without moving the ensemble mean.

Two properties make this a safe default. First, on continuous data nothing is revisited, no atom fires, and the projection vanishes identically — it is free when unused. Second, unlike a last-value spike it captures values that recur often but never consecutively (a rate that repeatedly returns to a round number). The pull of the level (its tendency to persist) is left to the trunk, where the random-walk candidate earns its weight by likelihood, so the projection remains a pure statement about mass, not mean.

Scoring convention. Because the `Dist` stays a Gaussian mixture, an "atom" is a narrow Gaussian, not a true Dirac: its standard deviation is scale-adaptive, $\max(0.005\,\sigma_D, 10^{-9})$ with σ_D the predictive's own std, its mass is the value's observed recency-weighted revisit frequency (total atom mass capped at 0.999), and a value qualifies only once revisited above a noise floor (1.8× the table's EMA rate), never on first sight. The density is therefore an ordinary Lebesgue density and every method is scored by the same rule. The convention matters for log-likelihood: at a revisited exact value the score grows like $\log(1/\text{width})$, so the projection is a bandwidth-dependent log-score machine on grid series. That is precisely why Table 1 restricts to *continuous* series (under 5% repeating changes), where the projection fires rarely and its contribution to the reported win-rates is immaterial. On grid-heavy series we measured the sensitivity directly: sweeping the atom width over two orders of magnitude (0.0005–0.05 of the predictive std, 400 grid-heavy series) moves the median held-out log-likelihood from +7.8 to +5.1 (off: +3.2) — the magnitudes are bandwidth artefacts, as the scaling predicts — while the per-series *winner* against **AutoETS** agrees on 91% of series between the extreme widths (win-rate 66% vs 57%), and CRPS, being bounded, barely moves (0.0035 vs 0.0053). Grid-series log-likelihood magnitudes should be read as qualitative; the orderings survive the bandwidth choice.

7 Coordinate learning

Whether a series is simple in a linear, log/multiplicative, or root coordinate is itself learnable. We add a coarse grid of Yeo–Johnson λ candidates — the signed Box–Cox family [Box and Cox, 1964, Yeo and Johnson, 2000], defined on all of $\mathbb{R}$ — to the pool, so the ensemble learns the coordinate online rather than committing up front. This is No-Free-Lunch-safe: a wrong coordinate is simply down-weighted by likelihood. The same mechanism expresses a non-negativity prior at $\lambda = 0$ (the log coordinate), though we note that because the `Dist` is a Gaussian mixture the inverse uses the delta-method linearisation, so non-negativity is approximate rather than exact.

8 Volatility and mean reversion as composable transforms

The same compositional vocabulary expresses conditional volatility and mean reversion without a separate forecaster. A **GARCH**-style scaling transform divides by a learned

conditional standard deviation; an Ornstein–Uhlenbeck transform (`ou_transform`) reverts the level toward a running mean at a learned speed; and a **GARCH**$(1,1)$-t terminal leaf (`garch_leaf`) gives the conform-last stage a conditional-variance recursion with Student-t tails. These are building blocks: `laplace` already carries an OU group in its multi-step ($k > 1$) candidate pool, and `laplace(leaf=garch_leaf)` swaps in the conditional-variance leaf for heavy-tailed series — no second named model required. (On asset-price/return series the right tool remains a fitted **GARCH**-t; see the price caveat of Section 10.)

These volatility recursions have a Bayesian reading. Each is a *score-driven* update [Creal et al., 2013, Harvey, 2013]: the **GARCH** variance recursion $h_t = h_{t-1} + (1-\delta)(y_t^2 - h_{t-1})$ is, term for term, the inverse-Fisher-scaled conditional score, and Hansen and Tong [2026] show via Tweedie's formula that under a conjugate prior and local precision discounting it is the *exact* Bayesian posterior-mean correction for the variance. The same analysis identifies the smoothing factor $\alpha = 1 - \delta$ that governs the exponential-smoothing and **GARCH**-style transforms of Section 3, and recovers the Gaussian-location update as the Kalman filter. Averaging the candidates therefore blends a family of (approximate) Bayesian volatility filters, a complementary justification for treating the clock itself as the object to learn.

9 Granularity: mixing time scales

Multi-step forecasting exposes one more axis a forecaster silently commits to: the *sampling granularity*. Natively, `laplace(k)` reaches horizon h by fanning its one-step model out h steps, and on long horizons that fan-out can inflate the predictive variance badly. A model running on a *decimated* clock — every s-th observation — reaches the same fine horizon h in only round(h/s) of its own steps, so its predictive stays tight; the price is blindness to the skipped points. Neither clock dominates: empirically (Section 10.3) the fine scale wins short horizons and the coarse scale long ones. The response is the library's usual one — do not choose. The `multiscale` wrapper — the default inside `laplace` whenever $k > 1$ (opt out with `scales=[1]`) — runs a full `laplace` on each of several decimated clocks (default strides $\{1, \lceil\sqrt{k}\rceil, k\}$) and, per horizon, mixes their predictive distributions with `Dist.combine` under likelihood softmax weights, so each horizon selects its effective granularity through the weights exactly as the pool selects coordinates or differencing orders.

Granularity is, however, the one candidate axis that cannot live *inside* the pool, which is why it is a wrapper rather than a transform. Decimation is many-ticks-to-one, breaking the scalar-in/scalar-out contract of `conjugate`; and a coarse-clock model emits no fine one-step predictive, so the trunk's one-step likelihood weighting (Section 4) has nothing to score it by. Two details matter. *Phase*: a fixed-phase stride-s model is up to $s-1$ ticks stale at forecast time, so the wrapper keeps s phase-shifted copies per scale, of which exactly one steps per tick — the freshest coarse forecast is always anchored at the current tick, at a per-tick cost of one base step per scale regardless of s. *Degeneracy*: horizon h draws only on scales $s \le h$, so at $k = 1$ the wrapper is exactly `laplace(1)` — multi-scale is free where it has nothing to add.

10 Experiments

Universe. To avoid a hand-picked series list we draw from the Federal Reserve Economic Data [Federal Reserve Bank of St. Louis], taking cached daily series and auto-transforming

each to its one-step *changes* (log-difference for strictly positive levels, else first difference) — the stationary-ish innovation stream where heavy tails and regime shifts live. The rule is part of the shared task definition: every method, ours and the baselines', forecasts the *same* change stream. Two caveats. Scale comparability of absolute log-likelihoods holds only on the log-differenced (unitless) subset — first differences keep the series' units, and a rescaling $y \mapsto ay$ shifts the log-density by $-\log a$ — so cross-series averages of absolute scores are indicative at best, and all win-rates and paired score differences are computed *within* series, where the Jacobian cancels. And because the log-vs-linear choice is a coordinate decision made before any model sees data, we checked its effect directly: rerunning the contest on 250 strictly-positive continuous series with plain first differences for every method moves `laplace`'s log-likelihood win-rates from 94/94/70% to 98/96/72% (vs **AutoARIMA**/**AutoETS**/**GARCH**-t) — the rule does not manufacture the edge; if anything it understates it. We keep the last 1000 changes per series and score the last 300 as a rolling one-step-ahead test window after a burn-in. Consistent with the scope set in Section 1, the evaluation universe is the *non-price* economic series: we classify each series a priori from its FRED title and exclude the asset-price classes (equity, foreign exchange, commodity), whose return series belong to the conditional-volatility regime and are treated separately in the price caveat below. The non-price universe is the general economic forecasting problem **skaters** is built for.

10.1 Against twelve distributional baselines

On the non-price universe we compare the general forecaster `laplace` against the distributional one-step baselines we could run on CPU at this scale. From **statsforecast** [Garza et al., 2022]: **AutoARIMA** and **AutoETS**. From the R forecasting ecosystem — the reference implementations, not Python ports — the actual Hyndman **auto.arima** and **thetaf** (the **forecast** package [Hyndman and Khandakar, 2008]; the Theta method [Assimakopoulos and Nikolopoulos, 2000] won the M3 competition), Svetunkov's **ADAM** auto state-space model [Svetunkov, 2023], a neural autoregression (**nnetar**), and a **rugarch GARCH**$(1,1)$-t [Ghalanos, 2022]. From **arch**, a constant-mean **GARCH**$(1,1)$ with Student-t innovations [Bollerslev, 1986, 1987], the classical heavy-tail / volatility-clustering state of the art. And four conformal opponents: **AutoARIMA** paired with split-conformal residual quantiles [Vovk et al., 2019] and its adaptive variant (ACI, Gibbs and Candès, 2021), plus conformal seasonal pools (CSP, Manokhin, 2026) in a fixed and a coverage-adaptive form, each KDE-smoothed into a density so it is scored on likelihood as well as CRPS.

The protocol is a *fair rolling one-step-ahead*: each baseline is fit on an expanding or rolling window with periodic refit, and every method — ours and theirs — is turned into the *same* `Dist` and scored on held-out log-likelihood *and* CRPS over the same test window. Parametric Student-t predictives (**arch** and **rugarch GARCH**, **ADAM**) are represented as finite Gaussian scale mixtures matching the analytic t density to $\sim 10^{-3}$; sample- or quantile-based emitters (**nnetar**, CSP) are kernel-smoothed into the same `Dist`. We report per-series win-rates (the fraction of series on which `laplace` scores better) on the 5,402 continuous series — those with under 5% exactly-repeating one-step changes — since on repeating/grid series the lattice projection (Section 6) gives a large but metric-specific advantage that would otherwise dominate the aggregate. Because many FRED families are panels (a yield curve, an index family) that are far from independent, we report each win-rate both *raw* and *family-weighted* (one vote per leading-letter family). Table 1 reports the result (the unified `study.py sota` run with `STUDY_EXCLUDE_PRICE`).

Baseline	LL (raw / fam.)	CRPS (raw / fam.)	Mean LL	N
AutoARIMA (**statsforecast**)	**87 / 87 %**	58 / 63 %	1.18	1688
AutoETS (**statsforecast**)	**95 / 94 %**	**77 / 78 %**	1.26	1688
auto.arima (R)	**84 / 85 %**	51 / 60 %	1.15	1918
thetaf (R)	**92 / 95 %**	**74 / 79 %**	1.24	1918
ADAM (R)	**98 / 97 %**	**80 / 82 %**	0.98	1918
nnetar (R)	**97 / 95 %**	**64 / 63 %**	−4.56	1918
GARCH$(1,1)$-t (**arch**)	51/50 %	46/48 %	1.69	1742
GARCH$(1,1)$-t (**rugarch**)	**90 / 91 %**	**60 / 62 %**	0.27	1918
AutoARIMA + conformal	**82 / 80 %**	36 / 49 %	0.28	1685
AutoARIMA + ACI	**84 / 81 %**	36 / 49 %	0.45	1685
CSP	**98 / 97 %**	**98 / 97 %**	−0.30	5402
CSP-adaptive	**98 / 97 %**	**98 / 97 %**	−0.09	5402

Table 1: Per-series win-rate of `laplace` versus each baseline on the **non-price** FRED universe — 5,402 continuous daily change-series (1,213 correlated families), from a 7,607-series sweep spanning daily, weekly, and monthly frequencies. Slower baselines cover a per-method subset under their compute budget (column N); `laplace` and CSP cover all 5,402. Win-rates are *raw* (per series) and *family-weighted* (one vote per leading-letter family, Section 11). LL counts series where `laplace` has the higher held-out log-likelihood, CRPS where it has the lower CRPS; bold marks a `laplace` win ($> 50\%$). "Mean LL" is the *baseline's* mean held-out log-likelihood (`laplace`: **1.56**; higher is better). Only **GARCH**-t (**arch**) escapes a `laplace` per-series likelihood win — a 50/50 coin-flip that Section 10.2 resolves by martingality.

Baseline	median Δ	mean Δ	fam. mean	95% CI
AutoARIMA (**statsforecast**)	+0.17	+0.63	+0.80	[+0.68, +0.93]
AutoETS (**statsforecast**)	+0.21	+0.55	+0.59	[+0.51, +0.67]
auto.arima (R)	+0.15	+0.58	+0.76	[+0.65, +0.88]
thetaf (R)	+0.18	+0.50	+0.57	[+0.49, +0.65]
ADAM (R)	+0.27	+0.75	+0.84	[+0.74, +0.94]
nnetar (R)	+1.60	+6.29	+7.88	[+6.97, +8.90]
GARCH$(1,1)$-t (**arch**)	+0.003	+0.11	+0.05	[+0.00, +0.11]
GARCH$(1,1)$-t (**rugarch**)	+0.38	+1.47	+1.49	[+1.27, +1.75]
AutoARIMA + conformal	+0.42	+1.53	+1.82	[+1.55, +2.12]
AutoARIMA + ACI	+0.37	+1.36	+1.72	[+1.44, +2.04]
CSP	+0.76	+1.86	+1.87	[+1.66, +2.11]
CSP-adaptive	+0.75	+1.65	+1.67	[+1.51, +1.86]

Table 2: Paired held-out log-score differences, $\Delta = \overline{\log p_{\texttt{laplace}}} - \overline{\log p_{\text{baseline}}}$ per series, on the continuous non-price universe (same coverage as Table 1). "fam. mean" weights one vote per leading-letter family; the interval is a 2,000-draw block bootstrap over families. Paired differences are scale-invariant (the Jacobian of the change transform cancels within a series), unlike the absolute "Mean LL" column of Table 1. Every median and every family-weighted mean is positive; against the **arch GARCH**-t the median is +0.003 — a genuine near-tie, resolved along the martingality axis in Section 10.2.

The likelihood race. On the non-price universe `laplace` wins the per-series held-out log-likelihood race against *eleven of the twelve* baselines by a wide margin — the **ARIMA**

Universe	N	LL win	median ΔLL	CRPS win
non-price continuous	1,742	51%	+0.003	46%
low-martingality tercile	580	78%	+0.182	78%
mid tercile	580	44%	−0.014	42%
high-martingality tercile	582	31%	−0.030	17%
asset price/returns	2,500	10%	−0.034	7%
with `garch_leaf`	2,500	18%	−0.031	5%

Table 3: `laplace` vs the **arch GARCH**$(1,1)$-t, by regime. "LL win" is the fraction of series where `laplace` has the higher held-out log-likelihood; ΔLL is the paired per-series log-score difference. Terciles split the continuous non-price universe by the martingality score m of Section 10.2 (low m = mean structure). The price rows are a 2,500-series stratified sample of the equity/FX/commodity classes (34 correlated families) under the identical one-step protocol: **GARCH**-t wins the per-series likelihood race 9 to 1, the martingale-heavy extreme of the same axis. Swapping in the library's **GARCH**$(1,1)$-t terminal leaf (`laplace(leaf=garch_leaf)`) roughly halves the paired deficit (mean ΔLL $-0.027 \to -0.015$) without closing it — and both `laplace` variants still beat **AutoARIMA** and **AutoETS** on 68% of the same price series.

family (both **statsforecast** and the real R **auto.arima**), the smoothing and state-space models (**AutoETS**, **thetaf**, **ADAM**), the neural autoregression (**nnetar**), the **rugarch** **GARCH**-t, and both conformal families — typically on 82–98% of series, and the win survives family-clustering. The lone exception is the **arch GARCH**$(1,1)$-t, a 50/50 tie treated next. Because win-rates discard magnitude and absolute mean log-likelihoods are not scale-comparable across mixed-unit series, Table 2 adds the natural aggregate for a proper scoring rule: the *paired* per-series log-score difference, whose Jacobian term cancels within a series. The paired view sharpens the claim: the median and family-weighted mean ΔLL are positive against all twelve baselines, and the **arch GARCH**-t contest is a genuine near-tie (median +0.003, family CI $[+0.00, +0.11]$) whose apparent mean-LL reversal in Table 1 (1.69 vs 1.56, computed over different coverage) is an artefact of absolute-score averaging that the paired column resolves — driven by the near-random-walk tail, which is exactly the martingality effect of Section 10.2.

GARCH-t, and the price caveat. **GARCH**$(1,1)$-t is the strongest opponent on log-likelihood, purpose-built for the heavy tails and volatility clustering of financial change-series. On the non-price universe it is the only baseline `laplace` does not clearly beat on likelihood — a 51/50% coin-flip — and that near-tie hides the real structure: more than for any other baseline, the **GARCH**-t contest is governed almost entirely by how *martingale-like* the series is, which we make precise in Section 10.2. The other half is the price regime. Restricting instead to the asset-price classes we excluded — equity, FX, commodity returns — reverses the verdict, and Table 3 makes the caveat quantitative rather than narrative. We make no general-purpose claim on price series and recommend **GARCH**-t there; this is No-Free-Lunch, reported as a split rather than hidden inside a pooled average.

CRPS. As ever, CRPS tells a mixed story. `laplace` beats the mean-model and state-space baselines on CRPS — family-weighted, **AutoARIMA** 63%, **AutoETS** 78%, **thetaf** 79%, **ADAM** 82%, and the density-emitting CSP 97% — but the split tightens against

the specialists: it is roughly even with the **arch GARCH**-t (48%) and *below* the fitted-mean conformal variants (49%), which are CRPS-optimised (or volatility-specialised) by construction. This is exactly the thesis of Section 5: likelihood is the metric where a faithful density wins; CRPS is what the conformal variants are constructed to optimise and where the volatility specialists are strongest, and there we are competitive but not dominant. A practitioner who truly only cares about CRPS can repoint the leaf (Section 11); a practitioner who cares about the density — for risk, sizing, or combination — should prefer the log-likelihood column.

Conformal prediction, specifically. The conformal opponents deserve a separate word; two kinds appear here. Conformal seasonal pools (CSP) sample a seasonal residual pool and KDE-smooth it into a density, so — unlike a raw conformal predictive system, whose cumulative distribution assigns zero density, and hence $-\infty$ log-likelihood, outside the observed residual range — CSP *can* be scored on likelihood. `laplace` beats it on both metrics (97% family-weighted on each), because a recency-blind seasonal pool tracks neither drift nor the volatility clock. Against the stronger *fitted-mean* conformal variants (**AutoARIMA**-mean residuals, split-conformal and ACI) the split is clean: `laplace` has the higher held-out log-likelihood on the large majority (80–81% family-weighted), while the conformal variants roughly match it on CRPS ($\sim 49\%$). That is exactly as it should be — finite-sample marginal coverage is conformal's actual guarantee, CRPS the score its interval-like output does well on, and it has no density to surrender on likelihood — and it is the empirical face of the pluggable-base argument: the right objective for *modelling* is the density, and the one place a downstream score should get a vote is the terminal leaf where the conjugation bottoms out. A user who wants CRPS-trained residual shaping can have it inside the same library by repointing that leaf to CRPS (Section 11) — a CRPS-fit density, not conformal's finite-sample coverage guarantee — without giving up the density the conformal systems cannot provide.

Scope. The non-price economic universe (5,402 continuous change-series of a 7,607-series sweep spanning daily, weekly, and monthly frequencies), one-step horizon. The price/return regime is reported as the explicit caveat above and is not part of the general-purpose claim; multi-step horizons are treated in Section 10.3; raw levels are left for later; zero-shot foundation models get their own protocol in Section 10.4.

10.2 The martingality gradient

The one baseline `laplace` does not simply dominate is **GARCH**-t, and the reason is not the price/non-price label per se but a property that varies *within* the non-price universe: how close each change-series is to a martingale. A pure martingale difference has no exploitable conditional mean — only its conditional *variance* can be modelled, which is exactly what **GARCH** specialises in; a series with mean structure (mean reversion, momentum) hands a forecaster signal that **GARCH**'s constant mean discards.

We quantify this with the Lo–MacKinlay variance ratio [Lo and MacKinlay, 1988], $\mathrm{VR}(q) = \mathrm{Var}\big(\sum_{i=1}^{q} \Delta_{t+i}\big)/\big(q\,\mathrm{Var}(\Delta_t)\big)$: $\mathrm{VR} = 1$ is a random walk, $\mathrm{VR} < 1$ mean reversion, $\mathrm{VR} > 1$ momentum. We score martingality $m = \exp(-|\ln \mathrm{VR}(5)|) \in (0, 1]$, with $m \to 1$ the pure random walk. Binning the daily-continuous non-price series into martingality terciles exposes the same axis in *every* contest, with a sign that reveals what each baseline models (Table 4). The **GARCH**-t (**arch**) contest is the extreme: `laplace` wins the held-out log-likelihood on 98% of the most mean-predictable series and only 5% of

Opponent (what it models)	low m	mid	high m
GARCH-t (**arch**) — variance	**98**	42	5
GARCH-t (**rugarch**) — variance	**100**	**86**	**73**
thetaf — trend	**98**	**92**	**69**
AutoETS — smoothing	**98**	**96**	**85**
ADAM — state-space	**100**	**100**	**99**
nnetar — neural AR	**100**	**99**	**97**
CSP, CSP-adaptive — conformal pool	**100**	**100**	**100**
AutoARIMA + conformal	**82**	**80**	**83**
AutoARIMA + ACI	**83**	**86**	**86**
auto.arima (R) — mean	**68**	**85**	**77**
AutoARIMA (**statsforecast**) — mean	**67**	**93**	**87**

Table 4: Per-series held-out log-likelihood win-rate of `laplace` (%) by martingality tercile, daily-continuous non-price series (n = 1,299, $\approx$ 433 per tercile); $m = \exp(-|\ln \mathrm{VR}(5)|)$ from the Lo–MacKinlay variance ratio, $m \to 1$ = random walk. Martingality modulates *every* contest, with a sign set by what the baseline models: for conditional-variance and trend models (top block) `laplace`'s edge *shrinks* toward the random-walk end; for **ARIMA** mean models (bottom) it *grows*; for the strong general and conformal opponents (middle) it is flat near 100%. **GARCH**-t (**arch**) is the sharpest case. Bold marks a `laplace` win ($> 50\%$).

the near-random-walks, so its aggregate ($\approx 50\%$) is not a close-run tie but the average of a decisive win and a decisive loss. The gradient is not unique to it, though. It runs the *same* way, more mildly, for every conditional-variance or trend model — the **rugarch** **GARCH**-t (100$\to$73%), **thetaf** (98$\to$69%), **AutoETS** (98$\to$85%) — because a random walk leaves only the variance to model, their comparative strength. It runs the *other* way for the mean models: against **AutoARIMA**, `laplace` wins *least* on the mean-predictable end (67%, where an **ARIMA** mean genuinely fits) and *more* toward the random walk (87%). And it is flat, near 100% throughout, for the strong general models (**ADAM**, **nnetar**, CSP). Martingality is thus a general organising axis — it sorts the opponents by what they model, variance specialists one way, mean specialists the other, generalists not at all — and **GARCH**-t is simply its sharpest case. The axis is not an artefact of the variance-ratio statistic. Binning instead by the absolute lag-one autocorrelation of the changes reproduces the same structure: against **GARCH**-t, `laplace` wins 27% of the least-autocorrelated tercile and 81% of the most; against **AutoARIMA** the gradient runs the other way (92% down to 78%); the generalists stay flat. We note the variance ratio is a second-order diagnostic — "martingale-like" here means low second-order mean predictability, not a complete characterisation of the conditional mean. This is also the mechanism behind the price caveat: price/return series are the martingale-heavy extreme of the axis, so the recommendation is precise — reach for **GARCH**-t not "on prices" but "on whatever is close to a random walk."

Robustness of the win. The likelihood win is not an artefact of one slice of the universe. It holds across observation frequency — `laplace` wins the log-likelihood race against every baseline on daily, weekly, *and* monthly non-price series — and across sequence length (from the 200-change minimum upward) and stickiness (the fraction

Method	h	**AutoARIMA**	**AutoETS**	**GARCH-t**	**Prophet**
single-scale	1	**75 %** / **58 %**	**81 %** / **69 %**	**66 %** / **59 %**	**87 %** / **75 %**
	5	**72 %** / **55 %**	**77 %** / **68 %**	**59 %** / 45 %	**76 %** / **58 %**
	20	**60 %** / 46 %	**66 %** / **59 %**	50 % / 36 %	**63 %** / 43 %
multi-scale	5	**80 %** / **62 %**	**85 %** / **74 %**	**62 %** / 49 %	**81 %** / **65 %**
(the default)	20	**73 %** / **53 %**	**79 %** / **64 %**	**60 %** / 38 %	**72 %** / 50 %

Table 5: Multi-step per-series win-rates (LL / CRPS) of `laplace` on 7,335 series of the non-price universe, 24 origins per series. Rows contrast the single-scale variant (`scales=[1]`, the native fan-out) with the multi-scale mixture that is the default at $k > 1$ (Section 9); at $h = 1$ the two coincide by construction and the multi-scale row is omitted. Bold marks a win ($> 50\%$). The single-scale fan-out decays with horizon — mean held-out log-likelihood $+2.65$, $+2.44$, $+1.10$ at $h = 1, 5, 20$ — while the granularity mixture holds it ($+2.70$, $+2.53$ at $h = 5, 20$), lifts every $h \geq 5$ log-likelihood contest by 3–13 points, and beats the single-scale variant head-to-head on 68% of series at $h = 20$ (59% at $h = 5$; exact ties at $h = 1$).

of exactly-repeating, zero-change steps, where the lattice projection of Section 6 contributes). An interactive companion recomputes every per-series win-rate and the accuracy/speed frontier live over any user-selected subset — sampled at random, balanced by category, or filtered by frequency, length, stickiness, or martingality — at `https://skaters.microprediction.org/robustness.html`.

10.3 Multi-step horizons and the granularity mixture

Multi-step is where the granularity axis of Section 9 earns its place. The target is stated exactly: at origin t and horizon $h \in \{1, 5, 20\}$, every forecaster emits a predictive for Δ_{t+h} — the *single* one-period change realised h steps later — not the cumulative h-period change and not the level. Each horizon's predictive is scored as the same `Dist` on held-out log-likelihood and CRPS at 24 evenly spaced origins per series. The opponents are the baselines for which a multi-step predictive is principled rather than improvised: **AutoARIMA** and **AutoETS** (`forecast(h)` from a periodically refit window), **GARCH**$(1, 1)$-t (its analytic h-step variance ladder), and **Prophet** [Taylor and Letham, 2018], whose fit-once-predict-h design this protocol matches exactly. The conformal variants are one-step constructions and are excluded.

Three findings (Table 5). First, the native fan-out decays with horizon: `laplace` wins the likelihood race against everything at $h = 1$ but slides toward a coin-flip against **GARCH**-t by $h = 20$, and its mean log-likelihood drops from $+2.65$ to $+1.10$ as the k-step variance inflates — severely so on a small set of series that dominate the (therefore unreported) mean CRPS. Second, a *fixed* coarse clock is not the answer: decimating to stride h and forecasting one step rescues the $h = 20$ density (mean $+2.43$) but throws away the intermediate points and loses to the mixture on 72% of series at $h = 20$ (75% at $h = 5$). Third, the likelihood-weighted mixture of scales recovers both ends: it beats the native fan-out on 68% of series at $h = 20$, leaves $h = 1$ untouched by construction (the wrapper passes scale one through), and wins every $h \geq 5$ log-likelihood contest. The gain is flat across autocorrelation terciles (63/59/63% vs native), consistent with its mechanism: it repairs the variance fan-out rather than exploiting mean structure. Granularity behaves like the other candidate axes — mixed under likelihood weights, the wrong scale costs only its softmax share.

Model	density	LL (all / cont.)	CRPS (all / cont.)	Mean LL (cont.)
Moirai (1.1-R-small)	native t	98 / **97** %	94 / 93 %	0.92
Lag-Llama	native t	67 / **99** %	65 / 94 %	0.39
Chronos-Bolt (small)	quantile*	76 / **100** %	67 / 88 %	−1.96
TimesFM (2.5-200M)	quantile*	75 / **100** %	58 / 72 %	−1.18

Table 6: Per-series win-rate of `laplace` versus four zero-shot foundation models on 120 change-series (69 continuous). *Chronos-Bolt/TimesFM log-likelihood is reconstructed from a quantile head and is tail-limited; read their CRPS as the fairer signal. `laplace` mean continuous logpdf: 1.51.

10.4 Against zero-shot foundation models

Pretrained time-series foundation models use a fundamentally different protocol: they are applied *zero-shot*, conditioning on a context window with no fitting. We evaluate four — Chronos-Bolt [Ansari et al., 2024], TimesFM 2.5 [Das et al., 2024], Moirai [Woo et al., 2024], and Lag-Llama [Rasul et al., 2023] — on 120 change-series (69 continuous). At each step the model receives a fixed 256-length window of the preceding changes and predicts the next change, with all windows for a series batched into one inference call. Each predictive is turned into the same `Dist` — the native Student-t head for Moirai/Lag-Llama, a sample/quantile reconstruction for Chronos-Bolt/TimesFM — and `laplace` is re-scored on the identical window. Table 6 reports per-series win-rates.

On the continuous series `laplace` beats all four foundation models zero-shot on both metrics (97–100% LL, 72–94% CRPS). The native-density models (Moirai, Lag-Llama) are the meaningful likelihood opponents and the closest, but still lose per-series. On repeat-heavy/grid series the foundation models are competitive or better — Lag-Llama even beats `laplace` on mean log-likelihood over the *full* universe (6.54 vs 4.52) by placing tight mass on revisited values, the same effect as the lattice projection — which is why the continuous split matters. We emphasise in the main text, not as a footnote, that this is a *narrow* comparison and not a broad foundation-model verdict: zero-shot, no refit, a fixed 256 context, and forecasting differenced streams rather than the levels these models were chiefly trained on — a protocol chosen because it is the one a practitioner with a single stream and no fitting budget actually faces.[1]

11 Ablations: repointing the objective

A companion sweep on 10,822 FRED series (every daily-tagged series by popularity) isolates the effect of the terminal-leaf objective against the **crepes** conformal predictive system (scored on its own CDF via the pinball decomposition of CRPS). `laplace` runs in its fixed default configuration on every series — no per-series tuning — while the opponent gets its best calibration window per series. Because this conformal opponent uses a naive (zero-change) mean, we read it as an ablation of our own objective knob rather than a state-of-the-art claim. Per-policy, de-correlating to one vote per correlated family (198 families):

[1] One exploratory check: naively fine-tuning Lag-Llama on a single series' history (5 epochs) overfit badly, collapsing held-out log-likelihood from +1.5 to below −100 at ∼15× the cost. We draw no general conclusion about fine-tuning from a single naive configuration — adapting a pretrained model to one short univariate stream is outside its design regime, and domain-level fine-tuning across many series remains untested here; the zero-shot result is the headline because zero-shot is the intended use.

Forecaster (identical structure)	CRPS, raw	CRPS, family	Mean LL
likelihood trunk + likelihood leaf	92.1 %	55.6 %	2.97
likelihood trunk + CRPS leaf	**96.6 %**	**64.8 %**	**2.99**
bare CRPS leaf (no trunk)	92.6 %	61.0 %	2.82

Table 7: Effect of the terminal-leaf objective (fixed `laplace` vs best-of-crepes, 10,822 series). The middle row is the `laplace` default (CRPS leaf, lattice projection on). A general likelihood policy already edges conformal on CRPS over independent families (55.6%); repointing only the terminal leaf to CRPS lifts that to 64.8% while slightly raising mean held-out log-likelihood in this sweep (2.97 $\rightarrow$ 2.99; a small difference we read as "no likelihood cost", not as a win), matching the dedicated CRPS specialist while modelling faithfully. Crepes' CRPS is U-shaped in window length (optimum near 250); `laplace` beats its best-of-$\{60 \ldots 1500\}$ on 96.4% of series, so the gap is structural, not a calibration-window choice.

Three further observations motivate the small public API. Priors wash out: across 25 series with ≥ 2000 changes the conventional prior-flavoured policies (trend, long-memory, drift, minimax, fast/slow) sit within 0.0006 nats of the uniform-prior general forecaster on the second half of each series, and the first-half winner beats the general forecaster on the second half only 44% of the time, worse than chance. Conform-last is near-free: on idealised light-tailed processes the CRPS leaf costs 0.005–0.007 nats versus the likelihood leaf, while on heavy-tailed and stochastic-volatility processes it helps (up to +0.04); real data lives in the second regime. And the lattice projection is free when unused. Together these justify exposing exactly one general forecaster (uniform prior, CRPS leaf, lattice projection and coordinate learning on by default), with volatility and mean-reversion left to composable transforms rather than separate named models; the remaining structural ideas do not pay their way out of sample.

12 Discussion

The unifying theme is No-Free-Lunch [Wolpert and Macready, 1997] taken seriously. Every "method" is a prior; averaged over all series none dominates. The correct response is not to choose one but to make the priors explicit, weight them online, and bound the cost of a wrong prior by how fast the weighting abandons it. Read that way, the named methods of classical forecasting are convenience bundles, and the natural object is a single adaptive ensemble with two orthogonal terminal knobs — *which proper scoring rule the leaf optimises*, and *whether a lattice projection is active* — neither of which is a prior over the trunk. The granularity mixture of Section 9 is the same argument applied to the sampling clock: which time scale a forecaster commits to is one more implicit prior, and the multi-step results show it too is better weighted online than chosen. The empirical headline supports the design: against the strongest distributional opponents available on CPU, including the heavy-tail and neural state of the art, the general forecaster wins the likelihood race, and where it does not lead — CRPS — the deficit is confined to methods explicitly optimised for that metric and is recoverable by repointing a single knob.

13 Heritage and reproducibility

skaters is a from-scratch rewrite distilling ideas from **timemachines** [Cotton, 2021], an earlier competition-tested online-forecasting package, and builds on years of live

distributional-prediction contests run on the microprediction platform [Cotton, 2022], where forecasts were scored continuously as full distributions. The conjugation recursion has a direct antecedent there: the platform's *multi-level distributional prediction* routed each stream into derived residual streams — the so-called *z-streams* — that were themselves predicted distributionally and composed back, "transforms all the way down" avant la lettre; **skaters** distils that platform mechanism into a single online, dependency-free abstraction. The library is implemented twice — pure Python and zero-dependency JavaScript — and a parity suite checks roughly 100,000 probe values to 10^{-6} on every run, so results are reproducible across both runtimes and in the browser via **Pyodide** [The Pyodide development team, 2021]. The benchmark of Section 10 is a single parallel harness (`benchmarks/study.py`, invoked as `study.py sota`) that scores every method through the identical `Dist` interface; given a FRED key and the optional baselines (**statsforecast**, **arch**, the R **forecast**/**smooth**/**rugarch** stack, and pure-`numpy` CSP) it reproduces Table 1 end to end, and `study.py sota summarize` regenerates the reported win-rates from the committed result file. The multi-step results of Section 10.3 have their own harness (`benchmarks/multistep_study.py`) under the same `Dist` protocol. All results in this paper were produced with the v0.11 defaults; from v0.12 the terminal leaf's scale-tracking rate default changed (`scale_alpha`: 0.01 → 0.03, a small out-of-sample improvement on both metrics), so pass `scale_alpha=0.01` to reproduce the reported numbers exactly.

References

Abdul Fatir Ansari et al. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024.

Vassilis Assimakopoulos and Konstantinos Nikolopoulos. The theta model: a decomposition approach to forecasting. *International Journal of Forecasting*, 16(4):521–530, 2000.

Tim Bollerslev. Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3):307–327, 1986.

Tim Bollerslev. A conditionally heteroskedastic time series model for speculative prices and rates of return. *The Review of Economics and Statistics*, 69(3):542–547, 1987.

George EP Box and David R Cox. An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, 26(2):211–243, 1964.

Peter Cotton. timemachines: Evaluation of online time-series forecasting algorithms. https://github.com/microprediction/timemachines, 2021.

Peter Cotton. *Microprediction: Building an Open AI Network*. MIT Press, Cambridge, MA, 2022.

Drew Creal, Siem Jan Koopman, and André Lucas. Generalized autoregressive score models with applications. *Journal of Applied Econometrics*, 28(5):777–795, 2013.

Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. *International Conference on Machine Learning*, 2024.

Robert F Engle. Autoregressive conditional heteroscedasticity with estimates of the variance of united kingdom inflation. *Econometrica*, 50(4):987–1007, 1982.

Federal Reserve Bank of St. Louis. Federal reserve economic data (FRED). https://fred.stlouisfed.org.

Federico Garza, Max Mergenthaler Canseco, Cristian Challu, and Kin G Olivares. StatsForecast: Lightning fast forecasting with statistical and econometric models. *PyCon Salt Lake City, Utah, US 2022*, 2022. Nixtla.

Alexios Ghalanos. **rugarch**: Univariate garch models, 2022. R package version 1.4-9.

Isaac Gibbs and Emmanuel Candès. Adaptive conformal inference under distribution shift. *Advances in Neural Information Processing Systems*, 34:1660–1672, 2021.

Tilmann Gneiting and Adrian E Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

Peter Reinhard Hansen and Chen Tong. Tweedie's formula and score-driven updating. *arXiv preprint arXiv:2605.15902*, 2026.

Andrew C Harvey. *Dynamic Models for Volatility and Heavy Tails: With Applications to Financial and Economic Time Series*. Cambridge University Press, 2013.

Jennifer A Hoeting, David Madigan, Adrian E Raftery, and Chris T Volinsky. Bayesian model averaging: a tutorial. *Statistical Science*, 14(4):382–401, 1999.

Rob J Hyndman and Yeasmin Khandakar. Automatic time series forecasting: the forecast package for R. *Journal of Statistical Software*, 27(3):1–22, 2008.

Rob J Hyndman, Anne B Koehler, Ralph D Snyder, and Simone Grose. A state space framework for automatic forecasting using exponential smoothing methods. *International Journal of Forecasting*, 18(3):439–454, 2002.

John L Kelly. A new interpretation of information rate. *The Bell System Technical Journal*, 35(4):917–926, 1956.

Andrew W Lo and A Craig MacKinlay. Stock market prices do not follow random walks: Evidence from a simple specification test. *The Review of Financial Studies*, 1(1):41–66, 1988.

Valery Manokhin. Conformal seasonal pools for distribution-free time-series prediction. *arXiv preprint arXiv:2605.03789*, 2026.

James E Matheson and Robert L Winkler. Scoring rules for continuous probability distributions. *Management Science*, 22(10):1087–1096, 1976.

Adrian E Raftery, Tilmann Gneiting, Fadoua Balabdaoui, and Michael Polakowski. Using Bayesian model averaging to calibrate forecast ensembles. *Monthly Weather Review*, 133 (5):1155–1174, 2005.

Kashif Rasul et al. Lag-Llama: Towards foundation models for probabilistic time series forecasting. *arXiv preprint arXiv:2310.08278*, 2023.

Danilo Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *International Conference on Machine Learning*, pages 1530–1538. PMLR, 2015.

David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):1181–1191, 2020.

Ivan Svetunkov. *Forecasting and Analytics with the Augmented Dynamic Adaptive Model (ADAM)*. Chapman and Hall/CRC, 2023.

Sean J Taylor and Benjamin Letham. Forecasting at scale. *The American Statistician*, 72 (1):37–45, 2018.

The Pyodide development team. Pyodide: Python with the scientific stack, compiled to WebAssembly. https://pyodide.org, 2021.

Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, 2005.

Vladimir Vovk, Ilia Nouretdinov, Valery Manokhin, and Alexander Gammerman. Conformal predictive distributions with kernels. In *Conformal and Probabilistic Prediction and Applications*, 2019.

BP Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962.

David H Wolpert and William G Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997.

Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. *International Conference on Machine Learning*, 2024.

In-Kwon Yeo and Richard A Johnson. A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4):954–959, 2000.